HOOX · AXIS · DOCUMENTATION

FRONTEND ENGINEERS · UX CONTRIBUTORS

UI MANUAL

Chart, editor, UI shell, indicators, results, store

Open charting PWA — own the axes, swap the engine

CONTENTS

01 Overview

02 Charting

03 Editor

04 Debugging

05 Ui Shell

06 Indicators

07 Results And Strategy

08 Store

OVERVIEW

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "UI" description: "AXIS UI subsystem: chart host, editor, UI shell, indicators, results, and Solid store—presentation without a closed engine."

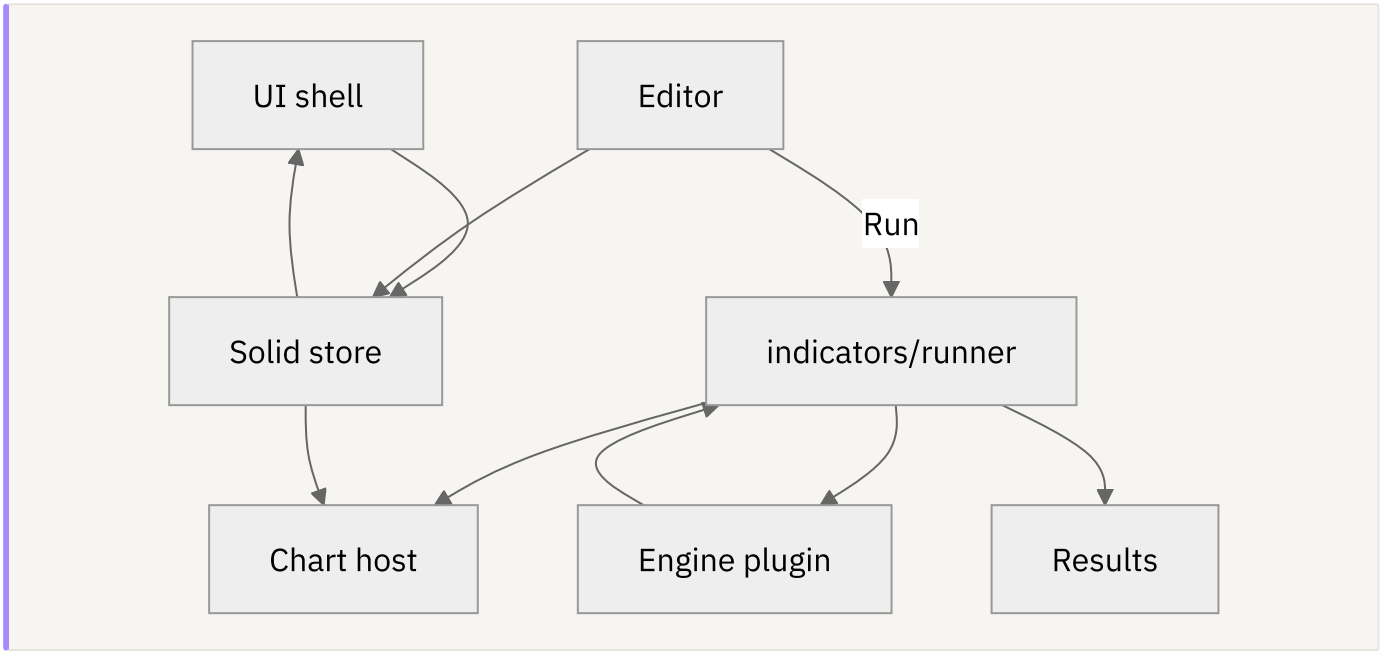
UI

UI is the AXIS browser presentation layer: everything the operator sees and clicks that is *not* language evaluation. Engines are plugins; AXIS applies their outputs to panes, markers, and drawers.

Abstract

Subsystem	Responsibility
Charting	Panes, multi-chart, replay, compare, drawings, badges
Editor	CM6, diagnostics, ruler, git bar, stats, dock/popout
Debugging	Scriptlogs, Profiler, Debug chips, Pins, Problems
UI shell	Topbar groups, docks side-by-side, palette, shortcuts, mobile shell, alerts
Scripts	Run pipeline, Scripts panel, overlay/sub-pane routing
Results & strategy	Event normalize, trades, equity, export
Store	Solid store, persistence, active plugins

Conceptual model



Invariant (AXIS ≠ engine): no Solid component imports PYNE directly; only engine modules talk to Python/HTTP runtimes.

Interface surface (visual map)

```
┌─ Topbar (grouped: market · data · compute · layout · panels · system) ─┐
├─ left docks ─┬─ ChartHost (1–4 slots) + drawings ─┬─ right docks ──────┤
│  watchlist   │  multi-pane + badges + replay       │  indicators|editor │
│  layers …    │                                     │  (side-by-side)    │
├─ Results / scriptlogs (bottom) ──────────────────────────────────────────┤
├─ System logs ─────────────────────────────────────────────────────────────┤
└─ StatusBar (connection HUD) ──────────────────────────────────────────────┘
   + Studio (Runtime · Wire · Settings) · Command palette (⌘K)
```

Implemented in `src/app.tsx` (repo root product UI).

AXIS UI: watchlist, multi-pane chart, and editor

Stack

Concern	Choice
Framework	SolidJS 1.9
Bundler	Vite 6 + `vite-plugin-solid`
Charts	lightweight-charts 5
Editor	CodeMirror 6
Icons	lucide-solid
Style	Tailwind 4 + void tokens (`data-theme`)

Relation to other tracks

- Operators: End User
- Structure: Architecture
- Contracts: Plugins
- Language: PYNE runtime

See also

- `LEGACY.md` — what is *not* the AXIS product path
- `TESTING.md` — unit/e2e around UI modules

CHARTING

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Charting" description: "AXIS chart: ChartHost, PaneManager, series factory, drawings, crosshair sync, and trade markers."

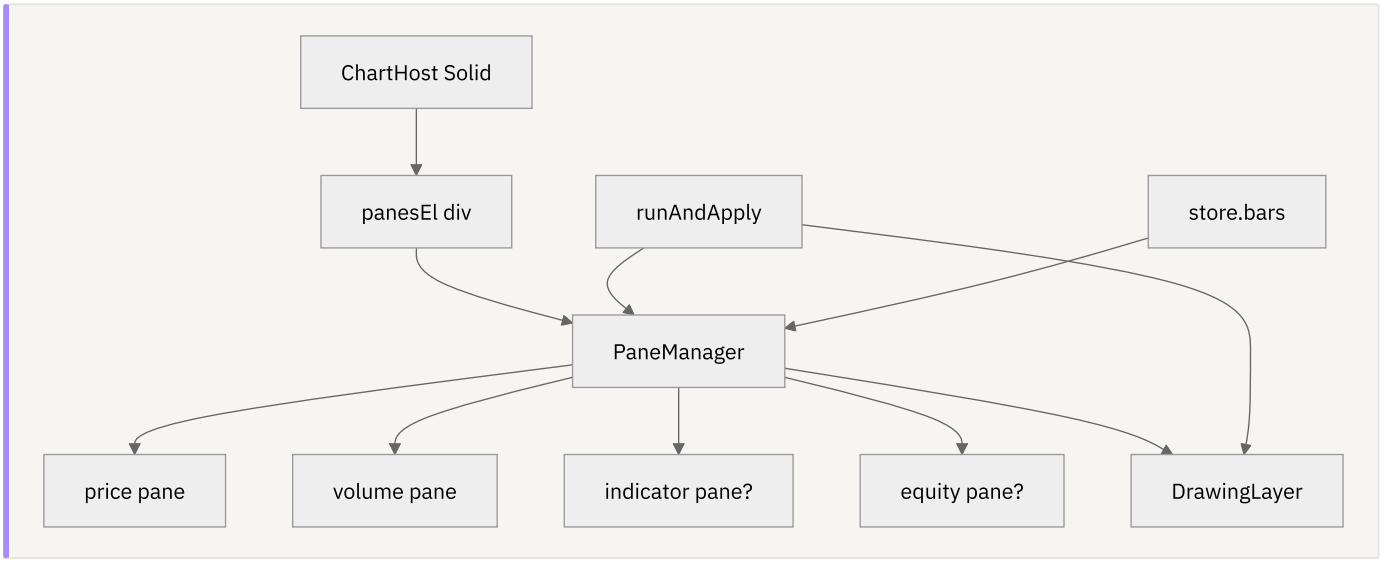
CHARTING

Charting is the spatial surface for OHLCV, overlays, and annotations. Implementation is **imperative** under a Solid shell so lightweight-charts owns its DOM subtree.

Abstract

Module	Role
`ChartHost.tsx` / `ChartWorkspace.tsx`	Solid mount; multi-slot grid; empty-state
`pane-manager.ts`	Multi-pane LWC charts, time sync, overlay apply
`pane-badge.ts`	Corner name + script action icons
`series-factory.ts`	Candle/volume/line series helpers + palette
`chart-type.ts`	Price style transforms (HA, hollow, …)
`bar-replay.ts` + `BarReplayControls`	History scrub / play
`layout.ts` / `layout-recipes.ts` / `chart-registry.ts`	Multi-chart layouts + presets
`compare-overlay.ts`	Second-symbol compare series
`volume-profile.ts`	OHLCV volume-at-price overlay
`drawing-layer.ts` + `drawings/*`	User + script drawings, templates, sync helpers
`price-precision.ts`	Price-scale decimals (`auto` or 0–8) from symbol + bars
`results/plot-visuals.ts`	Pine `plot.style_*` → LWC series kind (line, stepline, columns, area, …)
`crosshair-sync.ts`	Cross-pane crosshair
`pyne-drawings.ts`	Engine drawing objects → layer
`manager-access.ts`	Process-wide manager/layer getters

Conceptual model



Interface surface

Price, volume, and RSI panes with drawing toolbar

Panes

`store.panes`: `{ id, type, height, order, visible, label }`.

type	Typical series
`price`	Price series (style below) + overlay lines + markers + drawings
`volume`	Histogram
`indicator`	Non-overlay plots
`equity`	Strategy equity curve

Defaults: price + volume. Indicator/equity created on demand.

Price chart styles (`store.chartType`)

Topbar **Type** select (persisted). Implementation: `chart/chart-type.ts` + `createPriceSeries`.

chartType	Series	Notes
`candles`	Candlestick	Japanese candles (default)
`hollow`	Candlestick	Hollow body on up bars
`bars`	Bar	Classic OHLC bars
`heikinashi`	Candlestick	Heikin-Ashi transform of OHLCV
`line`	Line	Close only
`area`	Area	Close with fill
`baseline`	Baseline	Close vs first-bar base

Series still live under pane key `candle` so markers, hlines, and the drawing layer keep a stable host. Changing type swaps the LWC series and rebinds data without a full history reload.

Empty / status overlay

When `bars.length === 0`, ChartHost shows contextual title/sub from `store.status` (loading, error, running, or "Load data to begin").

Price-scale decimals

Price pane labels use `chart/price-precision.ts`:

Mode	Behavior
`auto` (default)	Merge symbol heuristics (e.g. BTC→2, SHIB→8) with recent OHLCV significant digits
`0` – `8`	Fixed display decimals + matching `minMove`

UI control cycles **A** → **0** → **…** → **8** → **A** on the price-pane scale chrome. Store field: `priceScaleDecimals` (persisted).

Plot style parity

Engine plot series honor Pine Script™ `plot.style_*` via `mapPlotStyleToSeriesKind` (`results/plot-visuals.ts` → pane-manager series factory):

Pine style (examples)	Chart series
`plot.style_line` / default	Line
`plot.style_stepline`	Stepline (LWC WithSteps)
`plot.style_stepline_diamond`	Stepline + vertex point markers
`plot.style_histogram`	Histogram (base 0)
`plot.style_columns`	Columns → LWC Histogram (base 0; no column-width/gap API)
`plot.style_area`	Area
`plot.style_circles`	Circular point markers + faint hairline
`plot.style_cross`	Larger point markers, connector hidden
`plot.style_linebr` / `steplinebr` / `areabr`	Broken line / stepline / area — `na` ends a segment (LWC would otherwise connect through whitespace)

LWC limits: Histogram series has no bar-width or gap option, so `columns` and `histogram` share the same bar geometry (both zero-based). Point markers are always circular — `cross` is approximated by discrete markers without a connector; `stepline_diamond` uses stepline + vertex markers. Sparse `plotshape` styles map diamond/cross to square markers (optional `+` / `×` glyphs when text is omitted).

`fill(plot1, plot2, color=…)` renders as SVG bands between the two series when both resolve on the same pane.

Multi-chart layouts

2×2 multi-chart grid with BTCUSDT loaded in the active slot

- Modes: **1 / 2H / 2V / 4** (`store.chartLayout`)
- Per-slot symbol/interval via registry; active slot drives topbar + Run
- **Named layouts** save/load (optional chrome snapshot)
- **Recipes** in Layouts menu: Scalp 1m+5m, Swing HTF, Quad, BTC majors (`layout-recipes.ts`)

Bar Replay

Bar replay strip on BTCUSDT: scrub, play, and speed Scrubbing and playing bar replay over loaded history

Topbar **Replay** → session over loaded OHLCV (`bar-replay.ts`):

- Enters with **full history visible** (cursor on last bar)
- Scrub / step / play; **Play at end restarts from bar 0**
- Mutually exclusive with Live

Compare & volume profile

ETHUSDT percent overlay compared against BTCUSDT
Layers panel: panes, volume profile overlay, scripts, drawings

- **Compare** control: second symbol as % or absolute overlay (`compare-overlay.ts`)
- **Volume profile** toggle in Layers → OHLCV approximation histogram (`volume-profile.ts`)

Pane badges

Each pane's top-left chip (`pane-badge.ts`): label + script actions when scripts are applied:

Icon	Action
Settings	Script inputs modal
File / code	Load that script's source into the docked editor
Eye	Show / hide series
Refresh	Re-run script
Trash	Remove from chart

Volume/equity without scripts: name + hide.

Screenshot

Module-bar **camera** (before Studio) captures Lightweight Charts panes to PNG:

Option	Meaning
Price / Panes / Workspace	Price only, all visible panes stacked, or stacked panes plus a header band
Drawings	Composite the SVG drawing overlay onto the pane
Watermark	`AXIS · symbol · interval · time` footer
Scale	1× native chart pixels, or 2×
Copy / Download	Clipboard `image/png`, or `axis-SYMBOL-TF-timestamp.png`

⌘K → **Screenshot chart...** / **Copy screenshot**. Last options persist in `localStorage`.

Drawings toolbar

See Drawings. Tools include cursor, hline, vline, trend, ray, extend, rect, ellipse, arrow, fib, measure, text (plus fib/shape/annotation/trading packs). The style bar edits color/width/dash plus per-tool extras (extend, fib levels, RR, text); last-used style is stored per kind. Templates and duplicate helpers live under Layers / drawings modules.

Crosshair & time

`syncTimeScales` / `syncCrosshair` keep multi-pane navigation coherent. `scrollToTime` / `jumpToDebugPin` used from Results and debug pins.

Internals

Solid vs imperative boundary

```
// ChartHost — Solid owns outer shell
// PaneManager only mutates panesEl — never put Solid children inside panesEl
```

On cleanup: destroy drawing layer, dispose manager, clear accessors.

Data path

`setDataToChart(bars)` applies OHLCV; `createEffect` on `store.bars` re-applies if bars change outside load helper.

Run apply (chart side)

From `runAndApply` (indicators):

- `syncOverlayLines` — update existing overlay series in place (avoids destroy→blank→add flash on live re-runs)
- Markers from normalized events (set in place)
- Script drawings atomic replace via DrawingLayer (`setScriptDrawings` without empty frame); `linefill` quads and `force_overlay` geometry route with other script drawings; `barcolor` tints candles via plot-visuals (not the SVG layer)
- Equity series when strategy report warrants (silent live runs skip hide thrash)

History vs live bars

- Full loads: `loadBars` → `chartDataGen++` → `setDataToChart` + fitContent
- Live: `multiplex` → `appendBar` + `manager.appendBar` only (no fitContent, no full setData)
- **Performance (2.0.x):** live tick path is O(1) append + conflation under load; heavy history paint is batched; load requests use **AbortController** so symbol switches cancel in-flight venue fetches
- Overlay re-runs prefer in-place series updates (`syncOverlayLines`) to avoid destroy→blank flash

series-factory

Centralizes series construction options, plot-style mapping, and color palette so Results plot summary and chart stay chromatically related.

Invariants

1. Solid reconciliation must not rewrite pane roots.
2. Script drawings ephemeral per run; user drawings store-backed.
3. Overlay routing: engine `meta.overlay` + script type; **oscillator-scale** series forced to sub-pane when they would vanish on price (see Indicators).
4. Shared non-overlay sub-pane id is stable: **`indicator`**.
5. Manager may be null before mount—runner no-ops chart apply safely.

Failure modes

Symptom	Cause
Blank panes after hot reload	Manager disposed; full remount
Overlays stack forever	Old path skipped removeOverlays
Fib/tools ignore clicks	No bars / time scale; wrong tool
Markers at t=0	Event times not bar-aligned

Worked example (dev)

```
getManager()?.scrollToTime(barTime);
```

See also

- Indicators
- Results and strategy
- End-user drawings

EDITOR

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Editor" description: "AXIS Pine editor: CodeMirror 6, diagnostics, 80-col ruler, git sync, stats, docked vs popout, editor bridge."

EDITOR

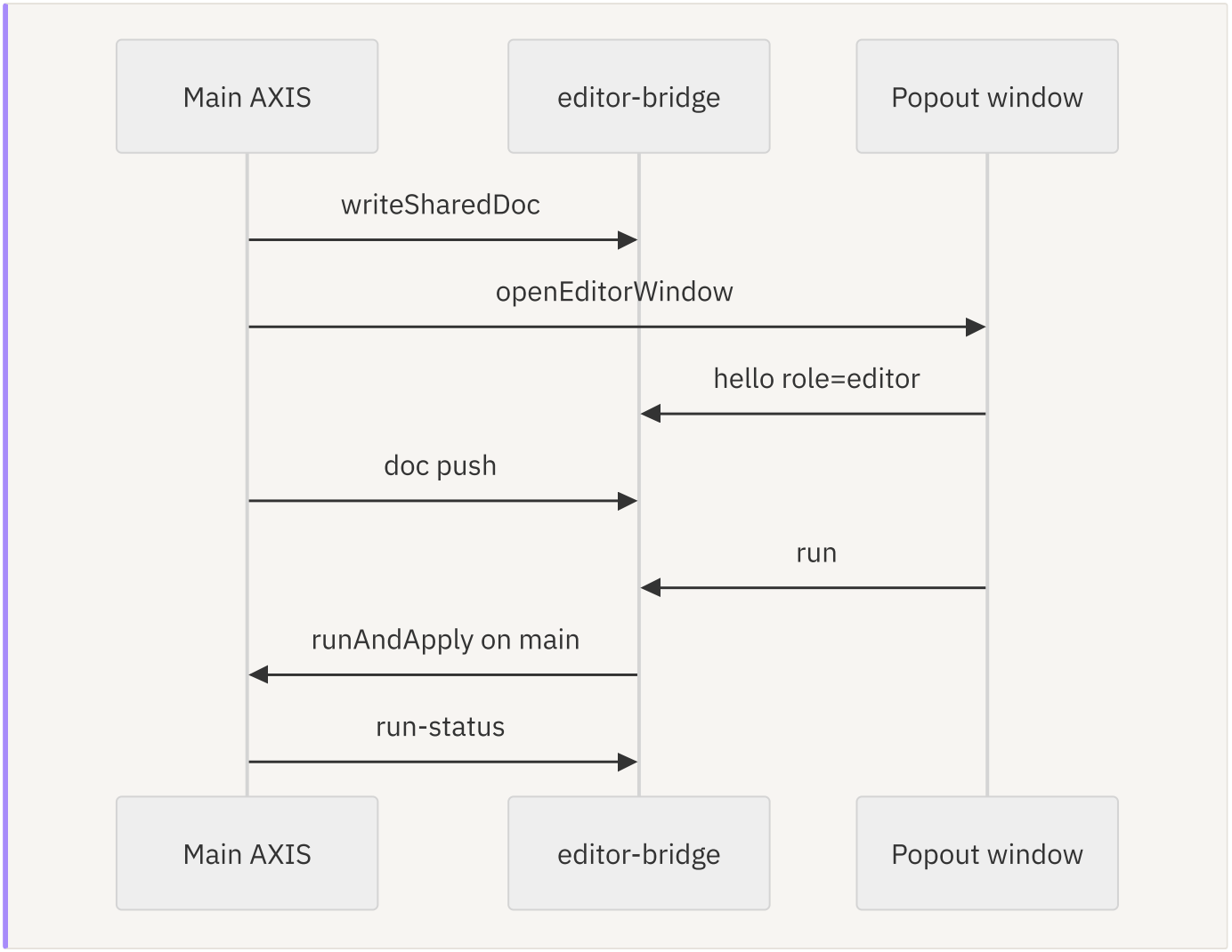
The editor is the **source axis of intent**: Pine text the operator authorizes the engine to run. Completion and hover can use local builtins metadata and optional remote LSP; **evaluation** is always an engine plugin.

CodeMirror 6 Pine editor showing a v6 RSI indicator

Abstract

Module	Role
`EditorPane.tsx`	Docked/standalone chrome, header tools, detach
`tabbed-editor.tsx`	Multi-tab docs, stats strip, Problems, Colors, Symbols, git bar
`PyneEditor.tsx`	CM6 mount (extensions + view APIs)
`pyne-language.ts`	StreamLanguage / highlighting (namespaces, types, multiline strings)
`pine-symbols.ts` + `SymbolEmojiManager.tsx`	TV-editor-safe glyph catalog + insert panel
`pyne-lsp.ts` + `data/pyne-builtins.json`	Offline completion/hover corpus
`column-ruler.ts`	80-character guide line
`minimap.ts`	Optional right-rail scroll overview
`function-outline.ts` + `FunctionOutline.tsx`	Optional right-rail function / type / enum tree
`diagnostics.ts`	Underlines, gutter, jump-to from last run
`editor-intel.ts`	Settings bag for lint / hover / complete / marks / timings
`inline-debug.ts`	EOL chips, pin gutter, line flash
`git-sync.ts` + `ui/EditorGitBar.tsx`	Pull / push / storage status
`ui/EditorProblems.tsx`	Collapsible problems list
`editor-bridge.ts`	Cross-window doc + run messages
`cm-void.ts`	Void-theme CM styling

Conceptual model



Why run on main? Chart, bars, and pane manager live in the main window. Popout is a pure editing surface.

Interface surface

Docked mode

- Right dock by default (`panelChrome.editor`); may sit **side-by-side** with other right panels (e.g. Indicators left of Editor)
- FloatableShell: dock left/right/bottom, float, new window
- Detach → popup or tab (`openEditorWindow`)

Popout mode

- Main shows “Editor detached” chip + **Reattach**
- Shared doc key + bridge messages: `hello` , `doc` , `run` , `run-status` , `reattach` , `popout-opened` , `popout-closed`

Run entry points

- Editor / topbar **Run** → `runAndApply(getDoc())`
- Popout Run → bridge → main executes

- Command palette: save library, git push/pull, jump to line, toggles

Header tools (`axis-editor-tools`)

Icon-only (labels in tooltips / `aria-label`). **Open in new tab** is in the panel-icon menu with dock options.

Control	Effect
Run	Run script against loaded bars
Scriptlogs	Open last-run `log.*` panel
Profiler	`profilerEnabled` + optional line % gutter; re-run when enabling
Debug	Inline end-of-line log/error chips (`inlineDebugEnabled`)
Pins	Chart markers + 📍 gutter (`debugPinsEnabled`); **Alt+P**
Ruler	80-column guide (`editorRulerEnabled`)
Overflow → Minimap	Right-rail canvas overview (`editorRightRail: minimap`); click/drag scrolls
Overflow → Function tree	Right-rail outline of functions, methods, types, enums (`editorRightRail: outline`); click jumps to the declaration. Mutually exclusive with Minimap
Panel icon → Open in new tab	Editor in a full browser tab (vs **New window** popup)

Details: Debugging.

Stats strip

Bottom of tabbed editor (`data-testid="axis-editor-stats"`):

Field	Meaning
Pos L:C	Cursor line:column (1-based)
Ln	Total lines in document
Words / Chars	Document stats
Diagnostics badge	Error/warning count from last run — click jumps to first
Problems	Expand/collapse problems list
Colors	Color chips / picker / converter for hex and `color.*`
Symbols	Insert TV-editor-safe arrows, box drawing, marks, spaces, and chart emoji (raw, quoted, or `plotchar`)
wrap	Soft wrap toggle (`editorWrapEnabled`, default on)

Symbols & emoji

Status-bar **Symbols** (`data-testid="axis-editor-symbols"`) opens a searchable catalog of glyphs used in Pine Script™ `plotchar` / `label.new` / table text.

- **Mono-safe** — ~1 cell in IBM Plex Mono / the TV editor (box drawing, `▲▼● ◆ ✓★`, arrows).
- **Wide / chart-only** — official `↑↓` and emoji (`🔥🚀📈`). Filter with **Mono-safe**.
- Click inserts at the cursor (replaces the selection). Right-click copies.
- Insert mode: raw glyph, quoted string, or a `plotchar(cond, "mark", "…", location.belowbar)` snippet.

Sources: TradingView® *Exploring Unicode*, Text and shapes, box-drawing U+2500, PineCoders conventions.

Highlighting

`pyne-language.ts` is a stateful stream tokenizer: `//@` annotations, `//` and `/* */` (including across lines), quoted and `"""` / `'''` strings with `\\n` escapes, `#RRGGBB(AA)` colors, namespaces (`ta.`, `label.new`), types (`int` / `float` / `string`, `series` / `simple` in qualifier position, and declared UDTs / enums as `typeName`, **bold** in the void theme), control vs definition keywords, and function names before `(`. Format leaves continuation lines of an open string or block comment untouched.

Four information surfaces

Surface	When	Answers
Hover	Rest on a symbol	Keyword, type, series, namespace, builtin, user input/function, or hex color
Param checklist	Inside a call	Used / current / unused args with type + default
Pre-eval	While typing / Save	What is wrong *before* Run
Debug chips	After Run, **Debug** on	Last-run `log.*` / errors on that source line

Hover + param checklist are writing-time docs — not a TradingView® debug API. Pre-eval marks the buffer; chips still read **last run**. Details: Debugging.

Diagnostics & Problems

Pre-eval (as you type) — `schedulePreeval` waits the **idle delay** after the last keystroke (default `PREEVAL_IDLE_MS` = 1000 ms; tab switch uses a shorter beat). Problems and underlines **stay** until the next idle check replaces them (optional **Clear marks while typing** hides them mid-keystroke). Local parse/lint (`localPreevaluate`) plus `POST /lsp/diagnostics` when engine=server and remote lint is on. Severity **error** disables Run when **Block Run on errors** is on (topbar, editor, Mod-Enter, command palette). Warnings and typos do not block.

Every lint / hover / complete / underline / chip option lives in **Settings** → **Editor** (command palette: *Open Editor Settings*). Defaults match the previous hardcoded behavior.

Local checks: unclosed `(` / `[` / `{` and strings, missing entry point, `study()` → `indicator()` / `strategy()`, bare `security()` → `request.security`, duplicate `indicator()` / `strategy()` / `library()`, unknown builtin members (`strategy.etry`, `plt` → "did you mean …?"). User functions and import aliases are not flagged.

Merge: remote style-noise C001–C004 is dropped; remote syntax errors win the same span; local unknown-member marks still show when remote is down. Hover tooltips use the diagnostic message (including the suggestion).

After a run, `diagnosticsFromLastRun(lastRun, doc)` still builds ranges from engine `error`, `meta.errors` / `diagnostics`, and error logs with line refs (`line 12:`, `L12:`, …). Live pre-eval for the current buffer is listed first; last-run engine messages are kept when they add detail (dropped only when the buffer no longer matches the stamped run source).

- CodeMirror: wavy underlines, severity gutter, hover tooltips
- **Problems** panel: clickable rows → `scrollToLine` / `jumpToDiagnostic`; origin **`pre-eval`** vs **`run`** when the diagnostic has a source
- Empty / pending pre-eval stays **No problems** (no flicker)
- Unit tests: `tests/editor-diagnostics.test.ts`, `tests/editor-problems.test.ts`, `tests/preevaluate.test.ts`

80-column ruler

`columnRulerExtension` paints a dashed guide at character column **80** (measured via CM coords). Toggle **Ruler** or command palette “Toggle Editor Ruler”.

Git sync bar

Tab toolbar **Pull / Push / status** (`EditorGitBar`):

- Uses active **storage** plugin (`local` | `cloud` | `git`)
- Push → `writeScript` (git commits when storage is git)
- Pull → optional plugin `sync('pull')` + list/reload bound library id
- Config stays in Settings / Script Library (no second credentials UI)

Completion corpus

`src/editor/data/pyne-builtins.json` is synced from PYNE (`scripts/sync-pyne-builtins.sh`). Offline completion/hover fall back to this catalog; optional remote LSP when the Pro API is available.

Library load

`loadLibraryDoc(doc, name)` on `editorRef` injects storage reads into the active tab.

Internals

editorRef pattern

App holds `{ getDoc, setDoc?, loadLibraryDoc?, scrollToLine?, jumpToDiagnostic?, getCursor?, insertAtCursor? }` populated by CM mount—avoids full-doc store writes every keystroke (doc also mirrored to `EDITOR_DOC_KEY`).

Persistence

Key / field	Role
`store.editor` + `panelChrome.editor`	Dock geometry, open, mode
`pynescript.axis.editor.doc`	Draft text
`profilerEnabled` / `inlineDebugEnabled` / `debugPinsEnabled` / `editorRulerEnabled` / `editorWrapEnabled`	Editor tools

Invariants

1. Empty doc Run is a no-op at topbar/editor handlers.
2. Popout does not own PaneManager.
3. Reattach restores shared doc into docked CM.
4. AXIS ≠ engine: editor never fully evaluates Pine itself; pre-eval is parse/lint only.
5. Debug/Pins still read **last run**; static diagnostics also come from pre-eval.

Failure modes

Symptom	Mitigation
Two editors diverge	Prefer bridge doc events; reattach
Detached but no window	Popup blocked; allow popups; reattach
Lost text on crash	editor.doc localStorage; library Save / git push
Blank CM (no lines)	Dock column height chain; editor must fill strip
No chips/pins	Enable Debug/Pins; logs need line and/or bar_index

See also

- Debugging — Debug vs Pins, Scriptlogs, Profiler
- UI shell — Topbar, docks, command palette
- Indicators
- Script library

DEBUGGING

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Debugging" description: "Scriptlogs, Profiler, Debug chips, chart Pins, diagnostics, and System Logs."

DEBUGGING

AXIS separates **script-authored diagnostics** (Pine `log.*`, engine errors, line timing) from **shell telemetry** (load, stream, engine connection).

Script Logs panel for last-run Pine log output

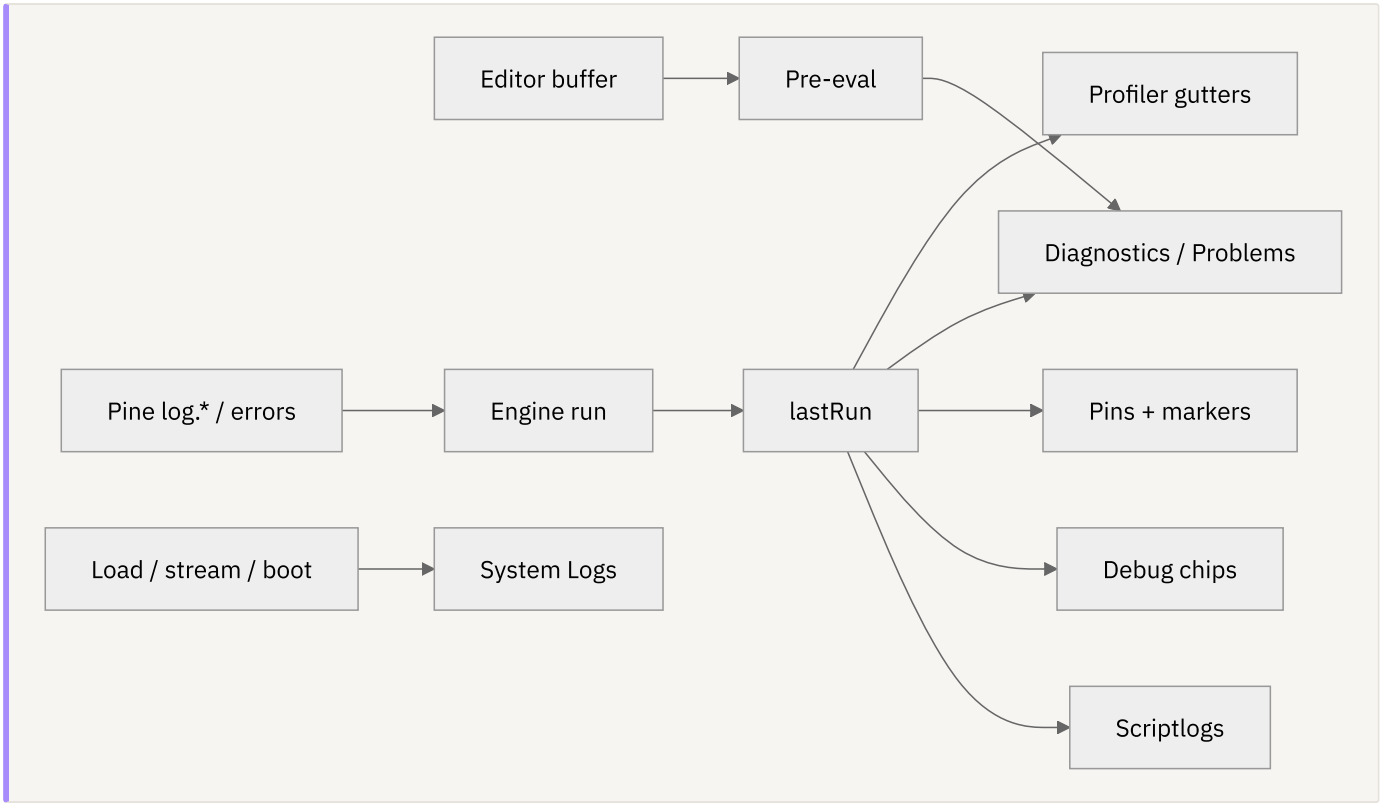
Writing-time and last-run info share four surfaces — hover, the param checklist, pre-eval, and debug chips — so each answer stays on its own chrome.

Abstract

Surface	Source of truth	Open / toggle
Hover	Builtin / identifier docs (local corpus or LSP)	Rest cursor on a symbol
Param checklist	Call signature + remaining named args	Type inside `plot(` / `ta.sma(` / …
Pre-eval / Problems	Parse/lint ∪ last-run engine errors	Underlines + Problems (`pre-eval` vs `run`)
Scriptlogs	`store.lastRun` → `normalizePyneLogs`	Editor header **Scriptlogs**
Profiler	`profilerEnabled` + `profile.lines`	Editor header **Profiler**
Debug chips	`inlineDebugEnabled` + last-run logs with **line**	Editor header **Debug**
Pins	`debugPinsEnabled` + logs with **bar_index / time**	Editor header **Pins** · **Alt+P**
System Logs	`store.logs` ring buffer	Status bar / logs strip

Module	Role
`ui/ScriptLogsPanel.tsx`	Floatable last-run `log.*` list
`results/pyne-logs.ts`	Normalize / filter / TSV
`results/profiler.ts`	Line % profile
`results/inline-debug.ts`	Annotations + pinable helpers
`results/debug-pins.ts`	Bar pins, markers, `countDebugPins`
`editor/inline-debug.ts`	CM chips, pin gutter, line flash
`editor/diagnostics.ts`	Error underlines / gutter / jump
`ui/EditorProblems.tsx`	Problems list UI
`chart/manager-access.ts`	`applyDebugPinsToChart` , `jumpToDebugPin`

Conceptual model



Invariant: Scriptlogs / Debug / Pins never write into `store.logs`.

Four information surfaces

Surface	When	Answers
Hover	Idle on a symbol	What is this builtin / identifier?
Param checklist	Inside a call	Which arguments remain?
Pre-eval	While typing / Save	What is wrong *before* Run?
Debug chips	After Run, **Debug** on	What did this **source line** log or error?

Hover and the param checklist never invent TradingView® host APIs — they read PYNE builtins / local docs. Pre-eval marks the buffer; Debug chips still read **last run** only.

Debug vs Pins

	Debug	Pins
Question	What happened on this **source line**?	Which **bar** was that on the chart?
Requires	Line ref (`line 12:` , structured `line` , …)	`bar_index` and/or `time` / `bar_time`
UI	End-of-line chips + optional level gutter	Chart circle markers + 📍 editor gutter
Click	Pin-able chips jump if bar info present	Chip / 📍 / Scriptlogs → chart jump + line flash
Store	`inlineDebugEnabled`	`debugPinsEnabled`

Workflow

1. Add `log.info` / `log.warning` / `log.error` (or rely on engine errors) with **line** and ideally **bar_index**.
2. **Run**.
3. Toggle **Debug** to see chips; **Pins** to mark bars (count shows on the Pins button).
4. Click a pin-able chip or 📍 → `jumpToDebugPin` (crosshair + scroll); source line flashes (`cm-debug-pin-flash`).

Both can be on at once. Pins without Debug still show 📍 gutter and chart markers.

Scriptlogs

Panel for **script** logging from the last run.

Pine call	Panel level
`log.info`	info
`log.warning`	warning
`log.error`	error

Open: Editor → **Scriptlogs** (`axis-btn-scriptlogs`). Pin-able rows jump to the chart when bar time/index is known.

Editor Profiler

Toggle **Profiler** (`axis-btn-profiler`):

- Persists `profilerEnabled`
- May send `profiler: true` to the engine
- Header shows last-run ms
- Gutter shows line `%` when `profile.lines` is present

Diagnostics & Problems

Independent of Debug chips: structural **error reporting** from pre-eval and the run payload.

- Pre-eval (`preeval*`) marks parse / lint issues as you type; last-run adds engine `error`, `meta.errors`, `diagnostics`, error-level logs
- CodeMirror underlines + severity gutter (larger hit target than line numbers)
- Stats strip badge (`axis-editor-diag-count`) and **Problems** list (`axis-editor-problems`)
- Each row shows origin **`pre-eval`** or **`run`** when the diagnostic has a source
- Click → jump to line/range
- Empty / in-flight pre-eval stays **No problems** (no checking flicker)

Scriptlogs vs System Logs

	Scriptlogs	System Logs
Source	Script `log.*` on last run	AXIS shell events
Store	`lastRun`	`store.logs`
UI	Floatable Scriptlogs	Strip above status bar
Control	Editor header	Status bar

Persistence & test ids

Key	Role
`panelChrome.scriptlogs` / `editor`	Panel chrome
`profilerEnabled`	Profiler
`inlineDebugEnabled`	Debug chips
`debugPinsEnabled`	Chart + pin gutter

testid	Role
`axis-btn-scriptlogs`	Open Scriptlogs
`axis-btn-profiler`	Profiler
`axis-btn-inline-debug`	Debug
`axis-btn-debug-pins`	Pins
`axis-debug-pin-count`	Pin count
`axis-editor-diag-count`	Diagnostics badge
`axis-editor-problems`	Problems panel

Unit coverage: `tests/pyne-logs.test.ts`, `tests/profiler.test.ts`, `tests/inline-debug.test.ts`, `tests/debug-pins.test.ts`, `tests/editor-diagnostics.test.ts`.

See also

- Editor
- Charting — markers and jump
- UI shell

UI SHELL

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "UI shell" description: "AXIS chrome: topbar, watchlist, status bar, settings, plugin manager, logs, and layout chrome around the chart."

UI SHELL

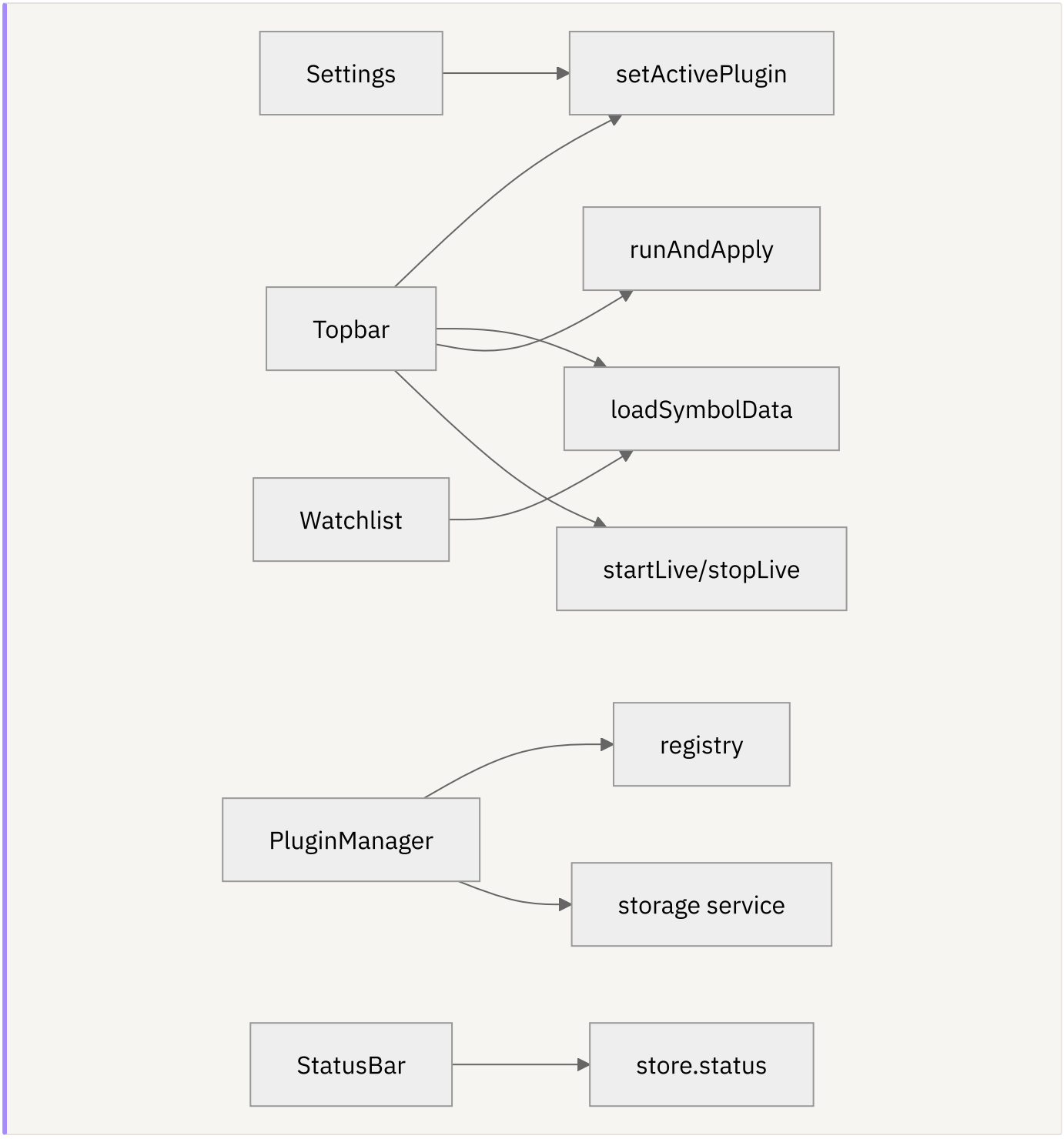
The shell is the non-canvas chrome: navigation of axes, market selection, modals, and status. It binds human gestures to store + plugins without owning calculation.

AXIS topbar groups: market, venue, engine, panels, Studio Command palette filtering theme commands

Abstract

Component	Path	Role
Topbar	`ui/Topbar.tsx`	Axes pickers, Load/Run/Live, theme, modals
Watchlist	`ui/Watchlist.tsx`	Symbols, quotes, click-to-load
StatusBar	`ui/StatusBar.tsx`	Status, engine/storage badges, trade snippet
StudioHost	`ui/studio/StudioHost.tsx`	Full-page Runtime / Wire / Settings / Workers / Plugins
SettingsPage	`ui/settings/SettingsPage.tsx`	Appearance, chart, live, keys, editor, theme
PluginsPage	`ui/plugins/PluginsPage.tsx`	Catalog / install / library
WirePage	`ui/wire/WirePage.tsx`	Compose-recipe wiring (source × stream × engine × storage × dataset)
SystemLogs	`ui/SystemLogs.tsx`	Ring buffer of `store.logs`
ResultsPanel	`ui/ResultsPanel.tsx`	Bottom drawer (see results doc)
TablesPanel	`ui/TablesPanel.tsx`	Bottom **Tables** tab per script (`table.*` moved off-chart; see drawings guide)
Icons	`ui/icons.tsx`	Lucide wrappers
ResizeHandle	`ui/ResizeHandle.tsx`	Panel widths/heights

Conceptual model



Topbar responsibilities

Grouped left → right (`axis-tb-group` , `data-tb-group`):

Group	Contents
brand	Logo + AXIS
market	Symbol, Interval, Type (`TopbarField`), Compare
data	Source (+ CSV upload), Load, Reload (icon-only)
compute	Engine, Stream, **Run**, Live, Replay
layout	Multi-chart layouts + recipes
panels	List, Editor, Library, **Data** (Data Source Manager), **Scripts**, Layers, Alerts, Data window, Inputs, Results
system	Second-row trailing: theme, fullscreen, chart-only, **screenshot**, Studio

Control	Effect
Symbol / interval	Store + Load (Enter / blur reload)
Source / Stream / Engine	`setActivePlugin` + side effects
Chart type	`setChartType` (candles, HA, …)
Compare	Second-symbol overlay
Load / Reload	One-shot historical fetch → chart
Data panel	Data Source Manager — background deep history
Scripts panel	Applied indicators/strategies list
Run	Editor doc → `runAndApply` ; **accent only while** `status === 'running'`
Live	multiplex start/stop (stops Replay)

Chart themes

Settings / command palette expose **ten** curated presets (void dark/light, classic, mono, obsidian, graphite, pacific, dusk, porcelain, parchment). High-end soft surfaces — not neon high-contrast. See `src/theme/presets.ts` . | Replay | Bar replay over loaded bars | | Panel toggles | `isPanelOpen` / dual-write chrome | | Settings / Plugins | Modal open | | Detach editor | Bridge + popout |

Integrated labels: `ui/TopbarField.tsx` (`.axis-tb-field*`).
`catalogTick` forces memo re-list when plugins install/remove.

Command palette

Ctrl/Cmd+K — `CommandPalette` + `command-registry.ts` : panels, theme, layouts, Run, symbol focus, editor toggles (ruler, debug, pins, profiler), jump to line, git push/pull, save library. Palette entries render any recorded shortcut chords.

Keyboard shortcuts

Global chord dispatch hub (`src/ui/shortcuts/`): capture-phase handling with a **dialog-skip guard** so shortcuts never fire while a modal is open. Wired to three scopes:

- **App-level** — palette, Run, panel toggles, replay, layouts
- **Chart** — keymap routed through the dispatch hub (cancel draft, layouts, replay)

- **Editor** — `src/editor/cm-line-ops.ts` `Prec.high` line ops: duplicate line, toggle comment, indent/unindent, delete line

Recorder and discovery:

- **Recorder**: Settings → Keyboard → record a chord; per-user overrides persist on the app store
- **Shortcuts modal**: lists default bindings with live rendering
- **Conflict flags**: scope-aware (`app.escape` vs `chart.cancel-draft` layer by design — no false conflict)

Alerts panel

Dockable **Alerts** (`src/alerts/*` + `AlertsPanel.tsx`): local-first book for **price**, **% change**, **drawing touch**, **indicator plots**, **Pine** **`alert()`** / **`alertcondition()`**, and on-chain kinds. Delivery: browser Notification, optional HTTPS webhook, optional L2 webhook. Live ticks evaluate price/drawing/plot kinds; strategy/indicator runs evaluate Pine `alert()` events (last bar). Continuous server-side arming is not required for session use.

Workspace snapshots

`storage/workspace-snapshot.ts` + optional chrome menus capture layout/plugin prefs for restore (distinct from OHLCV — bars still re-fetch on Load).

Panel docks (side-by-side)

Left/right docks with **2+ open panels** lay out **horizontally** (row), not stacked under each other:

- Example: **Indicators** left of **Editor** on the right strip
- Column width = **sum** of panel widths (capped)
- Each panel keeps its own width (resize handle on chart-facing edge)
- Bottom dock still stacks vertically

See `ui/panels/dock-layout.ts`, `FloatableShell.tsx`.

Mobile shell (phones / tablets)

`src/ui/mobile/MobileShell.tsx` + `src/ui/responsive.ts` — reactive `phone | tablet | desktop` mode from viewport + `pointer: coarse` (desktop ≥1024px is unchanged):

- **Phones (≤767px)**: compact header (brand · venue · symbol · interval) + bottom tab bar (**Chart · Panels · Editor · Studio · More**) replace the Topbar/StatusBar. Panels and More open as **nav overlays**.
- **Panel sheets**: docked/floating panels render as full-viewport sheets — one at a time, dismiss via **swipe-down** on the title bar. Desktop panel geometry is bypassed, never overwritten, so phone↔desktop switches preserve layouts. Phones boot chart-first.
- **Force-single chart**: 2H/2V/4 layouts collapse to the active slot with a slot-switcher chip row; other slots stay preserved in `store.chartLayout`.
- **Tablets (768–1023px)**: desktop shell with a horizontally scrollable topbar.
- Dialogs become **bottom sheets**; safe-area insets, `100dvh` root, ≥44px touch targets.

Watchlist

Watchlist rail with BTCUSDT selected and live 24h quotes

- Default universe includes majors (`BTCUSDT`, …)

- Named lists persist (`watchlist.lists` + `activeId`); legacy `symbols`-only state migrates to a **Main** list
- Create / rename / duplicate / delete lists from the panel header or the panel-icon menu
- Row bell creates a price alert (cross / above / below) at last
- Big-price rows (`ui/watchlist-row.ts`): direction tint + optional 24h change
- `refreshSec` clamped 5–120
- Source-aware labels (e.g. CSV mode)
- Click sets symbol and loads through active source

Extras (price card · ticker · time/price panels)

Optional floating widgets under the topbar (`ui/extras/`, toggled from the **Extra** menu):

- **Current price card** — live tick for the active symbol with trend tint over recent ticks.
- **Price ticker band** — marquee of watchlist symbols (seamless loop). Options: speed, direction, show 24h change, item spacing, band height, font size, opacity. Draggable when enabled; vertical offset resets from the menu. Symbol seed defaults to the watchlist (capped at 20).
- **Time / Price panels** — float-only clock (`timebadge.ts`: local/UTC + bar-countdown prefix) and big-price readouts docked under the topbar.
- State persists on `store.extras`; `anyExtraEnabled` drives the menu button state.

Panel motion

Panel open/close runs a short ease-out height/opacity pass (`ui/effects/registry.ts`, `FloatableShell` / `DockColumn`) — snappy, no layout jump.

Status bar

Connection HUD: LIVE, tick, engine, and source/stream/storage chips

Single row:

- **Left — Connection HUD** (`ConnectionHud`, `data-testid="axis-connection-hud"`)
  - Live control. The word stays Live. An amber dot means reconnecting; the tooltip carries attempt and backoff. Offline is the failure state.
  - Tick pulse (fixed-width price + age)
  - **Compose** chip — engine, source, stream, and storage. Opens Data or Studio.
  - **Diagnostics** expands the raw eng / run / mode / path / src / str / sto chips. The preference persists (`telemetry.hud.diagnostics`).
- **Right** — `status` / `statusMessage`, strategy snippet, bar / scripts / pane counts

Connect MCP is a command-palette and mobile More action, not a status-bar chip.

Routine “loaded N bars” and stream reconnect lines stay in System logs. A toast fires when retries are exhausted, or when the stream recovers from an outage longer than 8s.

Telemetry planes are ephemeral (`store.telemetry`). Diagnostics, compact HUD, and live prefs persist.

Live settings (Settings dialog)

Setting	Default	Effect
Enable live stream	on	`live.preferAfterLoad` → multiplex after Load and on boot when bars exist
Indicator re-run	every-tick	or `bar-close` only
Compact connection HUD	off	When diagnostics are on, hide source / stream / storage chips
Connection diagnostics	off	Status row shows Compose instead of raw chips

Logs

System Logs strip

- `appendLog(level, message, source)`
- Cap `MAX_LOGS` (500)
- Not persisted
- Boot messages, pyodide ready, live errors

Scriptlogs & Profiler

Script `log.*` output and editor line-timing live on a separate path — see Debugging. Controls sit on the **editor header** (not the topbar) and do not write into `store.logs`.

Theming

Settings → Theme: Void Dark preset and chart color tokens Switching chart theme presets

`store.theme` → `document.documentElement data-theme` (`dark` | `light`). Void dark is default product aesthetic. Settings → **Theme** picks a curated preset (void / classic / mono / boutique) and edits chart tokens live.

Accessibility / test hooks

E2E selectors use `data-testid` on critical controls (`axis-btn-load`, `axis-manager`, …) per `TESTING.md`.

Invariants

1. Shell never calls Python.
2. Plugin lists always come from registry facades (`listSources`, …).
3. Modal settings persist only on Save (not on every keystroke of endpoint field until save).
4. Live off leaves historical bars intact.

Failure modes

Symptom	Shell-side check
Pickers empty	Builtins not registered at boot
Live ignores click	Live is off (`stream: null`) or a multiplex error is in the logs
Settings probe fails	Network / CORS—not topbar bug

See also

- Manager and settings
- Store
- Architecture overview

INDICATORS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Scripts (indicators & strategies)" description: "AXIS applied-scripts UI: runAndApply pipeline, Scripts panel, overlay routing, and engine binding."

SCRIPTS (INDICATORS & STRATEGIES)

In AXIS, **scripts applied to the chart** may be Pine `indicator()` or `strategy()` definitions. The product panel is labeled **Scripts** (panel id remains `indicators` for workspace compatibility). The module boundary is `indicators/runner.ts` plus list UI—not the PYNE evaluator.

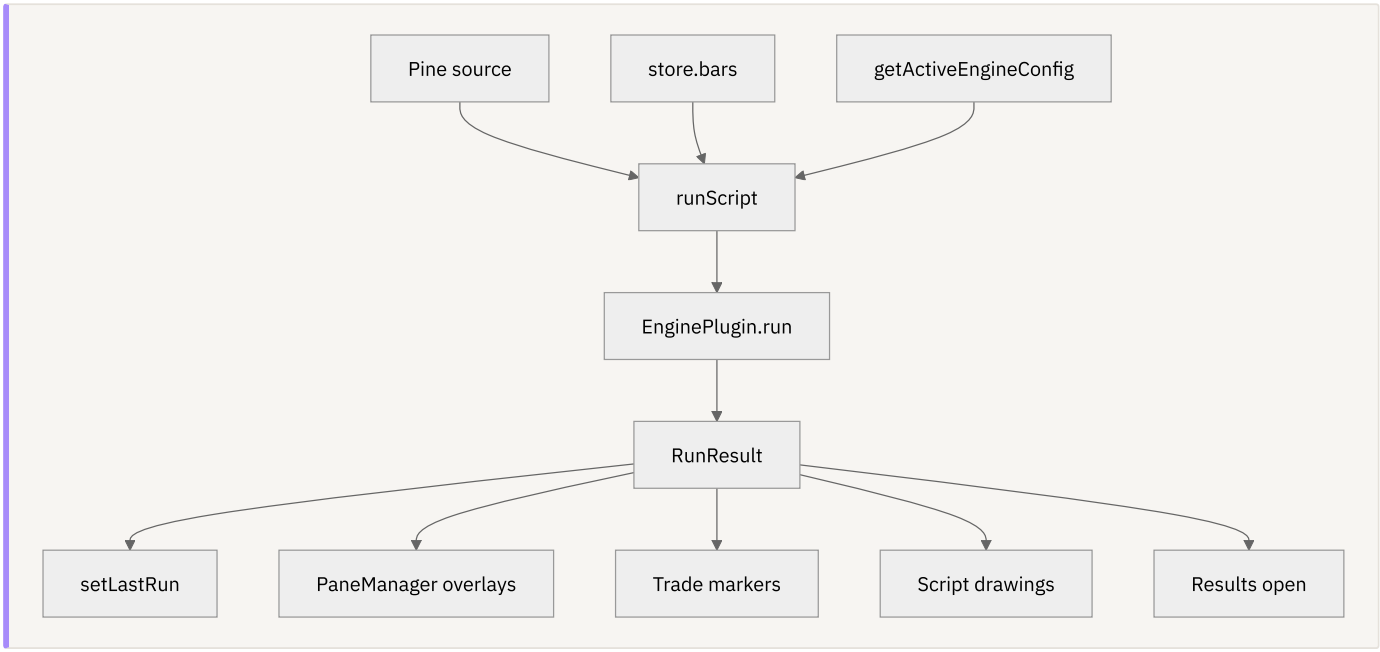
Scripts panel listing an applied RSI indicator

Abstract

Piece	Role
`runScript`	Engine invocation + status/timeouts
`runAndApply`	Chart + lastRun + results open; may `addIndicator`
`probeEndpoint`	Settings health check helper
Scripts panel (`IndicatorPanel`)	List / toggle / colors for `store.scripts`
`IndicatorCard`	Per-script UI chrome

Language semantics: PYNE runtime.

Conceptual model



Interface surface

runScript options

Option	Default	Meaning
`silent`	false	Quiet status; live re-run logging
timeout	derived	Interactive: scales with bars (60–180s); silent 45s

runAndApply options

Option	Default	Meaning
`openResults`	`!silent`	Open results drawer
`indicatorId`	optional	Update existing store indicator

Overlay routing

```
overlay = resolveOverlayFlag(meta.overlay, script_type)
  // explicit false → sub-pane
  // explicit true  → price
  // missing: indicator → sub-pane; strategy → price

if overlay && seriesWouldHideOnPrice(plots, bars):
  overlay = false   // RSI / scaled oscillators on BTC scale
paneId = overlay ? 'price' : `ind_<scriptId>`   // one sub-pane per script
```

Each non-overlay script gets its own sub-pane (`ind_<id>`), not a shared `indicator` host. Re-runs keep the script's private pane; legacy shared `indicator` panes migrate on re-run. Removing a script clears only that owner's series and destroys the sub-pane only when empty.

Persist + reopen

Applied scripts (`store.scripts` with full `code` , `paneId` , colors, inputs, strategy props) are written to localStorage with the rest of durable UI state. On boot they hydrate via `sanitizePersistedScripts` . After each successful history paint (`load-symbol` , DSM cache apply), `reapplyChartScripts` silently re-runs every visible script in plot-dependency order so the chart matches the saved set without manual re-run.

Detach path: `detachIndicatorFromChart` (Scripts panel, Layers, pane badges) — owner-scoped overlay clear, orphan pane sweep, no sibling wipe.

Prefer `indicator(..., overlay=false)` and raw `plot(rsi)` for oscillators — not `strategy(..., overlay=true)` with `rsi * 0.01` on price.

Series selection

Prefer `result.series` entries whose keys do not start with `_` / `__` . Fallback: single `plots[]` array as one line named from `script_name` .

Colors: `plot_meta[k].color` or `PLOT_PALETTE[i]` . Pine Script™ `plot.style_*` maps to LWC series kinds (line, stepline, histogram/columns, area, circles) — see Charting — plot style parity.

Indicator panel & pane badges

- Side list of `store.scripts` (`Indicator` : id, name, code, paneId, visible, plots)
- Visibility toggles affect display; re-run is still engine-driven
- Chart pane chips mirror settings / load source / eye / re-run / remove (see Charting)

Active engine binding

```
// plugins/active.ts pattern
getActiveEngine()        // registry lookup by activePlugins.engine
getActiveEngineConfig()  // pluginsConfig + store.endpoint merge
```

Switching engines mid-session does not auto-rerun; operator hits Run.

Live re-run coupling

Streams may set `live.needsRerun`; multiplex/runner path can call `runAndApply` with `silent: true` so the status bar is not spammed. Failures go to logs.

Internals

Path	Role
`src/indicators/runner.ts`	Pipeline
`src/indicators/IndicatorPanel.tsx`	List UI + built-in picker
`src/indicators/builtins/`	Original AXIS built-in Pine catalog
`src/plugins/active.ts`	Active engine/source helpers
`src/engines/catalog.ts`	Built-in engines
`src/results/plot-visuals.ts`	Plot style → series kind

Invariants

1. Runner is the **only** chart-facing calc orchestrator (avoid duplicate apply logic in components).
2. Errors still `setLastRun` with `status: 'error'` when possible.
3. Clearing overlays happens before re-adding—no unbounded series leak.
4. AXIS does not interpret Pine; it only maps `RunResult`.

Failure modes

Symptom	Layer
Engine exception	Results error + status
Success empty plots	Script/meta—not runner
Manager null	Chart not mounted; run still sets lastRun
AbortError	Timeout—reduce bars or optimize script

See also

- Charting
- Results and strategy
- Built-in scripts
- ADR-002 / ADR-004

RESULTS AND STRATEGY

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Results and strategy" description: "UI-side event normalization, trade pairing, metrics, equity curve, and ResultsPanel export pipeline."

RESULTS AND STRATEGY

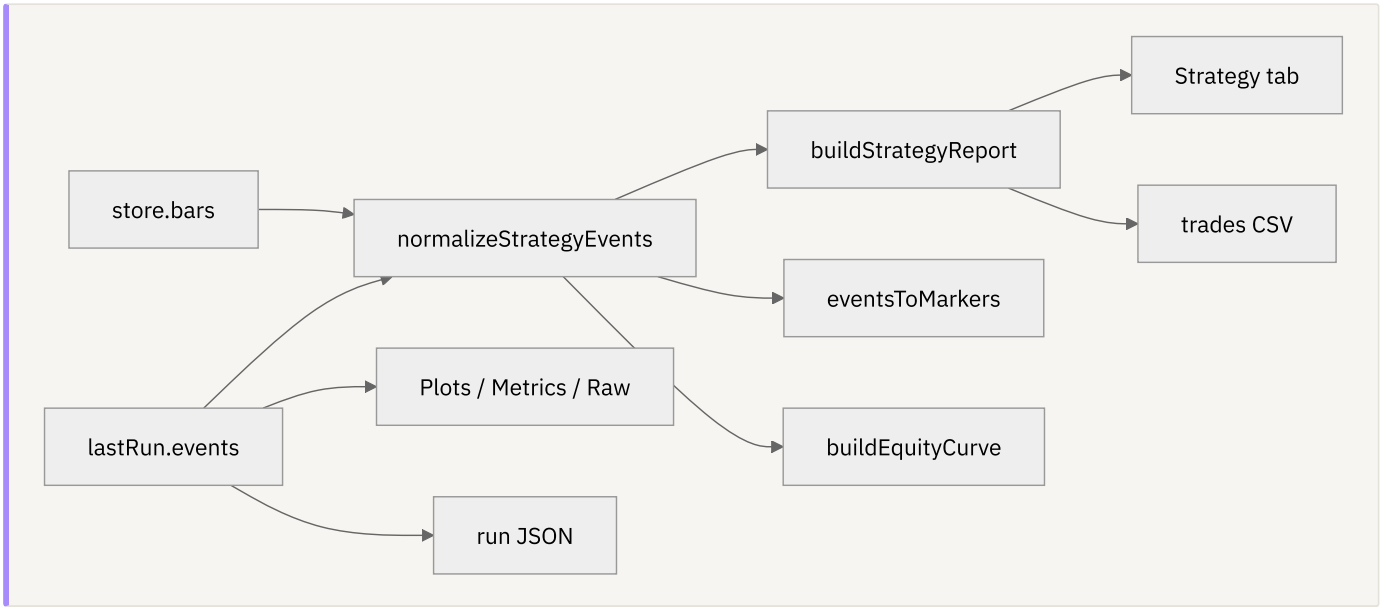
Results UI turns opaque engine payloads into **operator-consumable tables, metrics, and files**. It is a pure-ish viewer layer over `lastRun` + `bars`.

Results overlay with strategy stats, equity curve, and closed trades

Abstract

Module	Role
`results/events.ts`	Normalize heterogeneous event shapes; markers; equity series
`results/strategy.ts`	Shared walker `walkStrategyEvents` → trades, stats, positions, stream; CSV
`results/positions.ts`	Pyramiding-aware position cycles (avg-entry parity with the pyne broker, per-fill P&L)
`ui/ResultsModal.tsx`	Tabs, Events Stream/Open⇄Close switcher, export, jump-to-time
`ui/StrategyReport.tsx`	Stats cards (return %, bars-held, fills), equity curve, trades table, CSV
`ui/StatusBar.tsx`	Compact trade summary

Conceptual model



Event normalization

Engines may emit Pro API parity fields (`kind` , `bar_time` , `direction` , `ohlc`) or legacy UI fields (`type` , `time` , `dir` , `price`). Normalizer:

- Unifies time/price/dir/id
- Optionally fills price from bars when OHLC present/empty
- Can include order-like events for the Events tab

Trade pairing algorithm

Sketch (`buildStrategyReport`):

1. Sort by time.
2. On entry-like kind → open map by `id` .
3. On exit/close-like → match id, else sole open trade.
4. PnL = Δprice × long/short sign.
5. Aggregate win rate, profit factor, max DD on cumulative PnL.

Not modeled: fees, funding, partial fills, portfolio multi-symbol—viewer honesty.

ResultsPanel tabs

Tab	Data source
Events	`normalizedEvents` — **Stream** (chronological) or **Open⇄Close** (position cycles via `results/positions.ts`) view
Strategy	`report.trades` + `report.stats` (single column: stats → equity → trades)
Plots	`series` / `plots` summary
Metrics	meta + context + stats subset
Raw	`JSON.stringify(lastRun)`

Interactions: copy, export, `scrollToTime` on trade rows. Scriptlogs / Debug **Pins** also jump the chart (`jumpToDebugPin`) when bar time/index is present — see Debugging.

Equity

`buildEquityCurve` feeds chart equity pane when runner/strategy path requests it—derived series, not engine-native unless also provided.

Export (ADR-013)

Artifact	Function
`axis-run-*.json`	Full payload
`axis-trades-*.csv`	`tradesToCsv`
Clipboard	CSV when permitted

Invariants

1. Results recompute from `lastRun` reactively; they do not re-query engines.
2. Changing bars without re-run can change normalization fills—re-run after Load.
3. `lastRun` not rehydrated from localStorage (session artifact).
4. Infinite profit factor when no losses but profits exist—display must tolerate non-finite.

Failure modes

Symptom	AXIS explanation
Events > 0, trades = 0	Unpaired or missing prices
CSV disabled	No closed trades
Jump no-ops	Manager unmounted / invalid time

Internals cross-links

Operator narrative: Strategy and results.
Architecture: ADR-013.

See also

- Indicators
- Charting

STORE

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Store" description: "AXIS Solid store: AppState shape, activePlugins, persistence rules, logging, and alignment helpers."

STORE

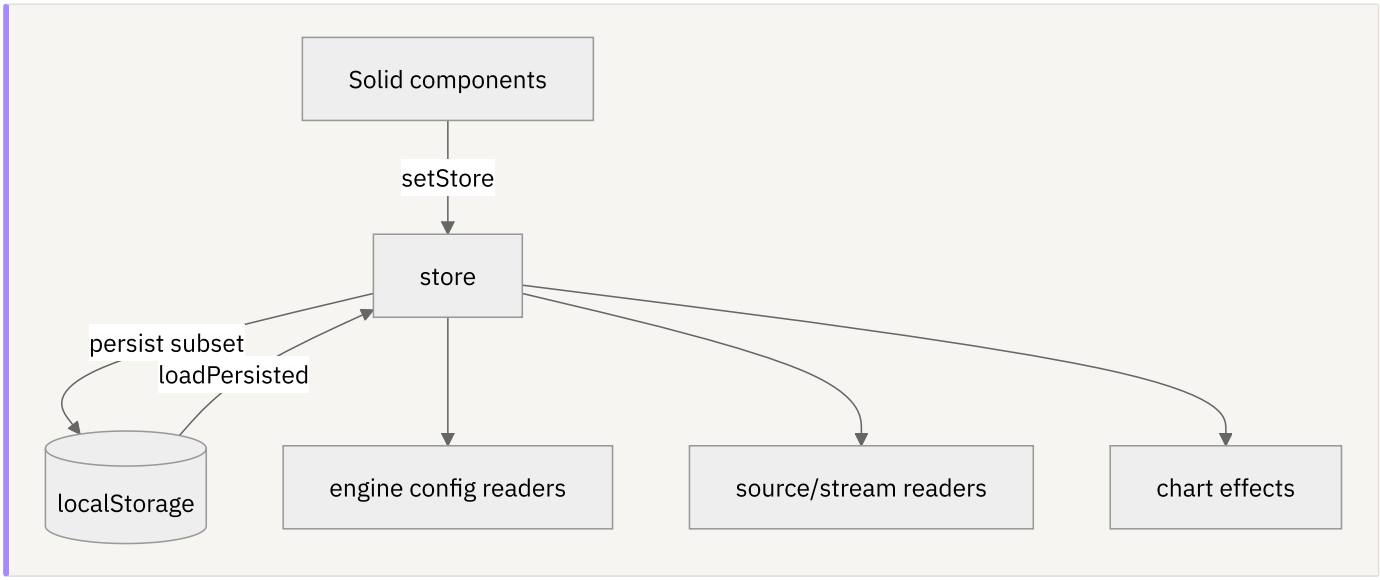
The Solid store is the **control-plane nexus** of AXIS. Plugins read configuration from it; UI binds reactively; persistence snapshots a safe subset to `pynescript.axis.v1`.

Abstract

Export	Role
`store` / `setStore`	`createStore<AppState>`
`persist`	Debounced localStorage write
`setActivePlugin`	Kinded selection + flat field mirrors
`appendLog` / status helpers	Operator-visible telemetry
`STORAGE_KEY`	`pynescript.axis.v1`
MCP	`src/mcp/dispatch.ts` reads/writes allowlisted store paths; snapshots redact secrets

Types: `src/store/types.ts`.

Conceptual model



AppState map

Field	Persistence	Notes
`bars`	no	Session history
`symbol`, `interval`, `exchange`	yes	Market context
`source`, `engine`	yes	Flat mirrors
`endpoint`	yes	Server engine base URL
`activePlugins`	yes	Canonical four-tuple
`pluginsConfig`	yes	Per-plugin fields
`scripts`, `panes`	yes	Indicator list + layout
`live`	partial	streamId important
`theme`	yes	dark/light
`editor`, `watchlist`, panels	yes	Chrome geometry
`stream.status`	yes/volatile	Connection hint
`status`, `statusMessage`, `lastRunMs`	yes*	UX; ok to restore
`lastRun`	**no**	Forced null on hydrate
`logs`	**no**	Forced empty
`profilerEnabled`	yes	Editor profiler (line % when engine provides profile)
`inlineDebugEnabled`	yes	End-of-line Debug chips from last-run logs
`debugPinsEnabled`	yes	Chart pin markers + editor pin gutter
`editorRulerEnabled`	yes	80-col guide (default **on**)
`editorWrapEnabled`	yes	Soft line wrap (default **on**; stats strip **wrap** toggle)
`panelChrome` / `alertsPanel` / `layerPanel` / `dataViewPanel`	yes	Dock chrome (side-by-side left/right)
`pineTablesLocation`	yes	Per-script Pine table placement (`chart` default / `bottom` Tables panel)
`chartLayout` / `savedLayouts`	yes	Multi-chart grid + named layouts
`compare`	prefs yes	Second-symbol overlay (bars ephemeral)
`drawingTool`	reset cursor	On hydrate
`drawings` / `drawingPrefs` / `drawingUi`	yes	User annotations + toolbar prefs
`selectedDrawingId`	**no**	Ephemeral selection

*Status fields may restore but boot also appends fresh logs.

Defaults (product)

Representative defaults from store:

Key	Default
symbol	`BTCUSDT`
interval	`1d`
source / active source	`binance-rest`
stream	`binance-ws`
engine	`server`
storage	`local`
endpoint	demo/API host or localhost depending on build era
editor width	460 docked open
watchlist	majors on **Main**, 15s refresh; named lists persist

Treat demo endpoints as **overridable**—operators should set local URLs in desk topology.

setActivePlugin

```
setActivePlugin(kind, id):
  activePlugins[kind] = id
  if source → source = id
  if engine → engine = id
  if stream → live.streamId = id
  persist()
```

Prevents registry/store split brain (architecture overview).

Logging

```
appendLog(level, message, source?)
levels: info | ok | warn | error
MAX_LOGS = 500
```

`setStatus` maps app status to log levels for important transitions.

Legacy path (removed)

The pre-Solid vanilla `state.js` shell was **removed in 2.21.0** (see legacy shell). Product AXIS uses the Solid store only — there is no dual-write target anymore.

Invariants

1. Hydrate never restores `lastRun` or `logs`.
2. Persist strips `bars`.
3. Key migration is read-repair (state namespaces).
4. `pluginsConfig` keys prefer `` `${kind}:${id}` ``.
5. Store is not a plugin registry—ids must exist in registry to function at runtime.

Failure modes

Mode	Behavior
JSON parse fail	Defaults
localStorage throws	No-op persist
Unknown plugin id	UI may show id; run/load fails at getActive*
Oversized drawings	Quota risk—export/clear

Testing notes

- Import `tests/setup.ts` for localStorage stubs
- Unit coverage includes store + migration patterns
- Do not require real IDB in pure store tests

See also

- State namespaces
- ADR-011
- UI shell